Colour lives in one file

Animated explainer · src/ui.js

The rest of the program describes what happened; ui.js decides how it looks. Every helper returns a string and nothing here writes to stdout, so the loop can still be tested with an injected logger.

8 scenes · one palette · zero stdout writes · NO_COLOR aware

CONTENTS

What is in this explainer

01 · Why this file exists 3

02 · The contract 4

03 · The line types 5

04 · Results 6

05 · Refusals 7

06 · The hook palette 8

07 · Colour level 9

08 · Recap 10

Page numbers refer to the folio at the foot of each page.

01 · Why this file exists

One palette. Every console line.

SRC/UI.JS · *source*

```
import chalk from 'chalk';

export const ui = {
  title: (text) => chalk.bold.cyan(text),
  step: (step, max) => chalk.blue('── step ' + step + '/' + max + ' ──'),
  tool: (name, args) => chalk.white('tool →') + ' ' + chalk.yellow(name) + ' ' + chalk.white(args),
  result: (text) => { /* ... */ },
  error: (text) => chalk.red(text),
};
```

Colour here — No other file imports chalk.

Strings only — Every helper returns text.

Testable — No stdout writes inside the loop.

> **remember** The loop narrates itself through these functions. It never knows a colour name.
> **On screen:** ui.js is the only file that imports chalk. Every helper returns a string, so the loop can log through any injected function. The whole palette can change here without touching agent.js.

02 · The contract

Return text. Never write it.

SRC/UI.JS · *source*

```
// Every helper returns a STRING and nothing here writes to stdout,
// which is why runTask can still take an injected log in the tests.
export const ui = {
  step: (step, max, width = 40) => chalk.blue('── step ' + step + '/' + max + ' ' + '─'.repeat(width)),
  think: (text) => chalk.cyan('think:') + ' ' + text,
  tool: (name, args) => chalk.white('tool →') + ' ' + chalk.yellow(name) + ' ' + chalk.white(args),
  error: (text) => chalk.red(text),
};
```

A function that returns a string can be tested, captured, piped or suppressed without changing the loop. **On screen:** The contract is simple: every helper returns a string and nothing here prints. That is why runTask can accept a custom log function and the tests can capture every line. Styling and I/O stay separate. The loop owns the narration, this file owns the paint.

03 · The line types

The agent has a visual language.

ONE STEP · *text*

```
── step 1/12 ──────────────────────────
think: Looking first: what is in the workspace.
tool → list_files {"path":"."}
result ← . 2 file(s)
  🛠️  [HOOK:PreToolUse] before every tool call - list_files {"path":"."}
  ⛔ FORBIDDEN SAVE ../escape.txt - path escapes the workspace
```

think — The model's narration.

tool — The name and raw arguments.

result — The first line of the observation.

FORBIDDEN — A refusal that must be impossible to miss.

> **On screen:** The console has a small vocabulary: step, think, tool, result, hook and refusal. Each line type has one colour and one job. The room can follow the run by shape alone. A refusal is bold red capitals, because it has to be findable from the back of a room.

04 · Results

A result explains itself.

SRC/UI.JS · *source*

```
result: (text) => {
  const line = firstLine(text);

  if (!isFailure(line)) {
    const out = firstStdout(text);
    const detail = out ? '\n↳ stdout: ' + out : '';
    return chalk.green('result ←') + ' ' + chalk.white(line) + detail;
  }

  const code = failedExitCode(line);
  const detail = code === null ? '' : '\n↳ exit ' + code + ': ' + explainExit(code);
  return chalk.red('result ← ' + line) + detail;
}
```

SUCCESS AND FAILURE · *text*

```
result ← exit code 0
         ↳ stdout: Hello from a tiny agent
result ← exit code 1
         ↳ exit 1: general failure - the command ran and then reported an error...
```

> **On screen:** A successful command shows its first stdout line, so exit code 0 is not a claim you have to trust. A failure shows the meaning of the exit code, not just the number. The result line is the model's observation and the human's explanation in one place.

05 · Refusals

Make the refusal impossible to miss.

SRC/UI.JS · *source*

```
forbidden: (action, requested, reason = '') => {
  const head = chalk.bold.red('FORBIDDEN') + ' ' + chalk.bold.white(String(action).toUpperCase());
  const why = reason ? ' ' + chalk.dim('- ' + reason) : '';
  return '  ⛔ ' + head + ' ' + chalk.red(String(requested)) + why;
}
```

THE LINE · *text*

```
  ⛔ FORBIDDEN SAVE ../escape.txt - path escapes the workspace: ../escape.txt
```

> Same stream as the hook trace. On stage the refusal is part of the narration, not an exception that scrolled past.
>
> **On screen:** forbidden is deliberately loud: bold red capitals, an icon, the action, the path and the reason. It goes to the same console stream as every other line. A refusal should be greppable in a log and visible from the back of a room.

06 · The hook palette

Hooks get their own colour vocabulary.

SRC/UI.JS · *source*

```
hookTitle: (label) => chalk.bold.cyan('── ' + label + ' ──'),
hookLine: (icon, name, colour = 'cyan', when = '', live = '') => {
  const paint = chalk[colour] ?? chalk.cyan;
  return '  ' + icon + '  ' + paint.bold(name.padEnd(16)) + ' ' + chalk.white(when) + live;
},
hookField: (label, value, colour = 'cyan') => {
  const paint = chalk[colour] ?? chalk.cyan;
  return '      ' + chalk.dim(label.padEnd(9)) + ' ' + paint(value);
},
hookTrace: (icon, name, colour = 'cyan', explanation = '', detail = '') => {
  const paint = chalk[colour] ?? chalk.cyan;
  return '  ' + icon + '  ' + paint.bold('[HOOK:' + name + ']') + ' ' + chalk.white(explanation) + ' ' + chalk.dim('- '
+ detail);
}
```

A HOOK FIRING · *text*

```
🛠️  [HOOK:PreToolUse] before every tool call - run_command {"command":"ls"}
```

> **On screen:** The hook catalogue and the live trace are also styled here. A colour name is passed as data, so hooks.js can stay a plain event table. An unknown colour falls back rather than throwing, which keeps the palette testable.

07 · Colour level

The environment decides how much colour.

- 🖥️ TTY chalk enables colour when output is a terminal.
- 🚫 NO_COLOR disables colour for accessible, plain logs.
- 💪 FORCE_COLOR forces colour when you need it.
- 📄 Piping a log file gets clean text automatically.

> **remember** The program does not decide whether colour is appropriate. The environment does.
> **On screen:** chalk handles TTY detection and the NO_COLOR / FORCE_COLOR conventions. ui.js inherits that instead of reimplementing it. Piping a run into a log file gives clean text for free.

08 · Recap

One file owns the paint.

• 1 palette

• 0 stdout writes

• 3 colour env rules

→

Return strings. Never write from a formatter.

→

One visual language. Step, think, tool, result, hook, refusal.

→

Explain failures. Exit codes get meanings.

→

Let the environment decide. TTY, NO_COLOR and FORCE_COLOR are honoured.

> **remember** Moving every colour into one file is what lets the loop stay about the loop.
> **On screen:** One palette, no stdout writes, and the environment decides colour level. Return strings, keep one visual language, explain failures, and let the terminal decide. That is why ui.js is small and why the rest of the code is readable.